
SAÉ 2.1&2

Deuxième livrable

Développement de l'interface graphique

1 Introduction

L'objectif de ce deuxième livrable va être de développer une interface permettant de jouer au jeu.

Vous trouverez sur eCampus une correction du premier livrable, vous êtes libre de vous baser sur le code du jour 1 d'un membre de votre groupe si souhaité.

Vous pouvez lancer le projet avec les commandes **mvn install**, puis **mvn javafx :run** dans le dossier **application**.

La correction a été adaptée au besoin d'un projet JavaFX : Toutes les propriétés écoutables sont devenues des **ObjectProperty**, **DoubleProperty**, **IntegerProperty**, etc. Vous pourrez modifier le code du modèle si cela vous semble utile.

2 Interface principale

Votre interface se décompose en cinq parties :

- Une zone centrale, dans laquelle on peut visualiser et jouer au jeu.
- Une zone, permettant de recommencer une partie, de voir les coups à jouer, d'afficher les coordonnées, changer les couleurs, etc.
- Une zone, permettant de gérer les différents paramètres de l'IA.
- Une zone supérieure, permettant de charger ou de sauvegarder une partie de jeu.
- Une zone affichant l'état du jeu en cours : Joueur actuel, phase en cours, nombre d'anneaux et de pions de chaque joueur.

De la même manière que gribouille, divisez votre interface en plusieurs fichiers FXML et donc en plusieurs contrôleurs. Le contrôleur principal contiendra toutes les informations dont plusieurs sous-contrôleurs dépendent.

Libre à vous d'agencer ces différentes parties de la façon dont vous le souhaitez, tant que les fonctionnalités de chaque parties sont respectées.

3 Les contrôleurs

3.1 Contrôleur principal

Le contrôleur principal doit être accessible depuis chaque sous-contrôleur. Créez une méthode `setMainController` ↪ (`MainController`) dans chaque sous-contrôleur et appelez là au moment du `initialize` du principal.

Le contrôleur principal doit stocker sous forme de propriété observable, toute information qui se partage entre différents contrôleurs.

De façon non exhaustive :

- La couleur d'un hexagone se configure grâce aux éléments du contrôleur de droite, mais est utilisé dans le contrôleur centrale pour l'affichage, c'est donc une propriété stocké dans le contrôleur principal.
- Le type d'affichage des coordonnées aussi
- Le mode d'interaction
- Une propriété pour savoir si un des joueurs a gagné
- Une instance du modèle
- etc.

3.1.1 Mode d'interaction

Notre application possédera trois états différents :

1. Un mode édition : On modifie l'état du jeu, suppression et ajout de pions et d'anneaux.
2. Un mode de jeu : On déplace un anneau
3. Un mode pour retirer une ligne : On choisit notre ligne et l'anneau à retirer du jeu

Pour répondre à ce besoin, créez une classe *abstraite* **InteractionMode**. Cette dernière a un accès au contrôleur principal et trois fonctions abstraites :

1. **handleClick(MouseEvent, HexSquare)**
2. **entered(MouseEvent, HexSquare)**
3. **exited(MouseEvent, HexSquare)**

Plus de détails sur ces différents modes dans la partie [3.3.1.3 : Les modes d'interactions](#).

3.1.2 Mise à jour du mode d'interaction

La mise à jour du mode d'interaction doit être faite de manière dynamique. Pour cela, commencez par créer dans le contrôleur principal une propriété permettant de stocker l'**InteractionMode** courant.

Il y aura deux sources d'actions qui pourront mener au changement du mode : On coche la case "mode édition" du contrôleur de jeu, ou lors du changement de l'état de jeu du modèle.

Pour créer un **Listener** se déclenchant dans les deux cas, utilisez la fonction **Bindings.createObjectBinding(Callable, Observable...)** qui permettra d'exécuter une fonction si l'un des **Observable** fourni en paramètre est modifié.

Pour le choix du mode d'interaction, le mode édition est prioritaire : Si la case est cochée, on est en mode édition, si elle ne l'est pas, on est en mode suppression de ligne si l'état du jeu en possède, sinon, on est en mode de jeu classique.

3.1.3 Vainqueur du jeu

De la même manière que pour le mode d'interaction, créer une **BooleanProperty** permettant de savoir si un des deux joueurs a gagné.

Si un des joueurs gagne, une **PopUp** affichant la couleur du gagnant s'affiche.

3.2 Contrôleur de jeu

Ce contrôleur va s'occuper de tout ce qui est lié au jeu et son affichage : mode édition, couleur des hexagones, taille des lignes, affichage des coordonnées, etc.

3.2.1 Les boutons

Une zone doit contenir quatre boutons : un permettant de lancer une nouvelle partie aléatoire (fonction *random-Game* dans **IFactory**), et chacun des autres permettant de charger un état généré par une fonction des **Factory**.

3.2.2 Contrôle du mode édition

Une zone doit posséder une checkbox permettant d'activer ou désactiver le mode édition.

Si le mode édition est activé, il doit y avoir une sous-zone qui s'active (désactivée si le mode n'est pas activé), permettant de choisir deux choses : Si on veut créer un pion ou un anneau ainsi que son équipe.

Pour cela, vous créerez deux **ToggleGroup**, chacun d'entre eux lié à deux boutons : noir/blanc ou anneau/pion.

3.2.3 Contrôle des couleurs

Une zone doit pouvoir permettre de choisir la couleur associé à une catégorie de case. Vous utiliserez pour cela des **ColorPicker** et par défaut les couleurs devront être respectivement : gris, gris foncé, gris clair.

3.2.4 Affichage des coordonnées

Une **CheckBox** doit pouvoir permettre de désactiver ou d'activer l'affichage des coordonnées directement dans les cases de jeu.

Une **ListView** doit pouvoir permettre de switcher entre l'affichage en mode **Cubique** et en mode **2D**.

3.2.5 Epaisseur des bordures

Un **Slider** doit permettre de pouvoir modifier entre 1 et 5 l'épaisseur des bordures des hexagones du terrain.

3.3 Contrôleur central

La zone centrale ne contiendra qu'un seul **Pane** sur laquelle vous allez disposer l'ensemble des cases du terrain grâce à une spécialisation de la classe **Polygon**.

3.3.1 Représentation du terrain

3.3.1.1 Classe HexSquare

Créez une classe **HexSquare** qui héritera de la classe **Polygon** de JavaFX. Cette dernière représentera une case hexagonale du terrain. Chaque instance possédera :

- Une propriété **Color** provenant du contrôleur principal ;
- La **Coordinate** représentée par cette case ;
- Un attribut **Shape** permettant de représenter soit un **Pawn**, un **Ring** ou un **Circle** permettant de mettre en avant que c'est une case accessible ;
- Un **Label** correspondant à sa coordonnée.

A la création d'une instance, vous utiliserez la fonction **to2DCoordinate** de la coordonnée associée pour connaître la position visuel du centre de l'hexagone, puis vous allez ajouté les différents points grâce à la méthode **getPoints().add** ou **getPoints().addAll**. Les différents points de l'hexagone par rapport au centre sont situés :

1. **En haut à gauche** : $-LARGEUR/2, -HAUTEUR/4$
2. **En haut** : $0, -HAUTEUR/2$
3. **En haut à droite** : $+LARGEUR/2, -HAUTEUR/4$
4. **En bas à droite** : $+LARGEUR/2, +HAUTEUR/4$
5. **En bas** : $0, +HAUTEUR/2$
6. **En bas à gauche** : $-LARGEUR/2, +HAUTEUR/4$

Avec **LARGEUR** et **HAUTEUR** deux variables arbitraire représentant la taille d'une case.

Les événements **OnMouseClicked**, **OnMouseEntered**, **OnMouseExited** doivent appeler les méthodes correspondantes de l'**InteractionMode** actuel.

Par défaut, le survol d'une case lui fera prendre une couleur différente, et le fait de quitter la case remettra la couleur de base.

La mise à jour de la **Shape** stockée doit supprimer ou ajouter l'élément correspondant dans le pane.

Créer le **Label** associé à cette case et ajouter le aux éléments du **Pane** centrale.

3.3.1.2 Création du terrain

Au moment du paramétrage du contrôleur principale dans le contrôleur centrale, vous allez pouvoir récupérer l'état du jeu et créer le terrain. Chaque case du terrain de jeu doit avoir son **HexSquare** associé et stocker dans une **HashMap** permettant de faire le lien entre une **Coordinate** et un **HexSquare**.

Vous penserez à passer la bonne propriété couleur à chaque **HexSquare** de telle sorte à ce que aucune case n'est une case adjacente de la même couleur.

3.3.1.3 Les modes d'interactions

Mode édition

En mode édition, le clic droit supprimera l'élément situé sur la case s'il y en a un, sinon il ne fera rien.

Le clic gauche lui, se basera sur les deux **ToggleGroup** du contrôleur de jeu ([partie 3.2.2](#)) afin de savoir s'il doit créer un pion ou un anneau et son équipe. Si la case occupée possède le même élément, il est supprimé, sinon il est remplacé par l'élément créé.

Mode de jeu classique

En mode de jeu classique, le premier clic sur une case occupée par un anneau doit faire apparaître sur les cases accessibles par ce dernier, un élément visuel permettant de savoir qu'on peut s'y déplacer : Un cercle, un point, un carré, etc.

Au deuxième clic, si la case ciblée est bien accessible, on déplace l'anneau sélectionné précédemment sur la case en question.

Mode retirer une ligne

Comme pour le mode de jeu, cela se passe en deux temps : sélection de la ligne à supprimer, puis de l'anneau.

Lors du survol d'une case, on regarde si la coordonnée associée appartient à une ligne stockée dans l'état de jeu courant. Si c'est le cas, on met en surbrillance la ligne entière.

Au premier clic, si on clic sur une case qui appartient à une ligne, cette dernière se verouille avec une couleur spécifique puis on passe à l'anneau. Si une case est présente dans deux lignes différentes, on prendra en priorité la première que l'on voit possédant la case.

Au deuxième clic, si on est bien sur un anneau, on retire la ligne + l'anneau et on passe à la suite. Si on clic sur un anneau dont la couleur est différente de la ligne, rien ne se passe.

3.4 Contrôleur IA

3.4.1 Contrôle des scores

Une zone doit contenir 4 **TextField** permettant de modifier les scores des différents points évalué par l'IA : Nombre d'anneau, victoire, ligne de 4, etc. Chaque **TextField** doit être associé à un **Label** permettant d'identifier la caractéristique auquel il est lié.

3.4.2 Affichage de l'évaluation

Pour afficher l'état de la situation actuel, vous allez utiliser une **ProgressBar**. Par défaut, son fond sera blanc et son remplissage devra être noir, plus la barre est noir, plus les noir auront l'avantage. Par défaut elle sera remplie à 50%.

A chaque changement d'état de jeu, on calcul le score actuel et on fait un pourcentage situé entre le score minimal (l'adversaire de l'IA a gagné) et le score maximal (l'IA a gagné), puis on remplit la **ProgressBar** avec ce pourcentage pour afficher qui a l'avantage.

3.4.3 Coup à jouer

Un bouton devra faire en sorte qu'une **PopUp** apparaisse affichant le coup qu'elle considère le plus efficace.

Pour vous faciliter l'affichage, vous pouvez redéfinir les **toString** des sous-classes de la classe **Action**.

3.5 Contrôleur du bas

En bas de votre application doit apparaître en continu le tour du joueur en cours (Noir/Blanc), ainsi que l'état l'**InteractionMode** de votre application.

3.5.1 Contrôleur menu

Un menu doit être apparent en haut de votre application.

Deux **Menu** sont attendu :

- **A propos** : affiche les informations relatives à votre groupe
- Un charger/sauvegarder de la partie en cours

3.5.1.1 Sauvegarde et chargement

La persistance des données est un aspect essentiel pour permettre d’interrompre et de reprendre une partie. Vous devez implémenter un mécanisme de sauvegarde et de chargement au format texte ou binaire (selon votre modèle), respectant une structure stricte.

Vos fichiers devront avoir une extension qui se démarque, par exemple : **.yns**, **.yin**, etc.

Sécurisation du fichier : Le « Magic Number »

Pour éviter que l’application ne tente de lire un fichier corrompu ou un document d’une autre nature (ce qui provoquerait un plantage du programme), vous devez intégrer une sécurité au tout début du fichier.

À l’image des bases de données **SQLite** dont les 16 premiers octets contiennent toujours la signature hexadécimale 53 51 4c 69 74 65 20 66 6f 72 6d 61 74 20 33 00 (qui se traduit par la chaîne de caractères *"SQLite format 3"*), votre format de fichier devra débiter par un identifiant unique, communément appelé un **Magic Number**.

Vous utiliserez comme signature les 6 premiers octets représentant la chaîne : SAE212.

Structure attendue du fichier

Le fichier devra être structuré de la manière suivante :

1. **En-tête (Sécurité)** : La chaîne SAE212.
2. **État global** : Le joueur dont c’est le tour (Noir ou Blanc) et la phase actuelle du jeu, pour optimiser on pourrait ne stocker qu’une lettre.
3. **Données du terrain** : Une liste ordonnée des coordonnées occupées, spécifiant pour chacune s’il s’y trouve un pion ou un anneau, ainsi que sa couleur.

Processus de chargement

Lors du clic sur le bouton de chargement, vous devez utiliser un **FileChooser** de JavaFX avec un filtre pour ne pouvoir sélectionner que votre extension. Avant de décoder le contenu :

- Lisez les premiers octets.
- Si le *Magic Number* ne correspond pas à SAE212, interrompez immédiatement la lecture et affichez une **Alert** d’erreur à l’utilisateur indiquant : « Format de fichier invalide ».
- Si la signature est valide, poursuivez la lecture, reconstruisez l’état du modèle, puis forcez la mise à jour des contrôleurs.